

Workflows de MailingAI — detalle por nodo

Documentación de los 5 workflows de esta carpeta: qué hace cada uno y, dentro de cada uno, qué hace cada nodo. Para instrucciones de instalación/importación ver el `README.md` de la raíz del proyecto.

Convención de nombres: el prefijo numérico indica el orden de uso, no de importación (todos se importan juntos). `00` es un subworkflow interno que llaman `01` , `02` y `03` — nunca se ejecuta solo.

<code>00</code> - Graph Fetch (subworkflow, uso interno)	<- llamado por <code>01</code> , <code>02</code> y <code>03</code>
<code>01</code> - Fetch Sent Items	<- punto de entrada más común
<code>02</code> - Fetch Message Series (parametrizado)	<- alternativa con filtros libres
<code>03</code> - Fetch Related Thread	<- requiere un <code>conversation_id</code> ya guardado
<code>04</code> - Generate Activity Charts	<- se corre después de tener datos (<code>01/02/03</code>)

00 — Graph Fetch (subworkflow, uso interno)

Archivo: `00-mailingai-graph-fetch-subworkflow.json`

No se ejecuta manualmente. Recibe parámetros de otro workflow vía el nodo `Execute Workflow` , llama a Microsoft Graph, normaliza cada mensaje al esquema `mailing.messages` y hace upsert. Es el único lugar del proyecto donde se llama a Graph API — así los workflows `01/02/03` no repiten esa lógica.

Parámetros de entrada (definidos en el trigger, ver nodo 1):

Parámetro	Tipo	Uso
<code>folder</code>	string	Nombre de carpeta de Graph (<code>SentItems</code> , etc.) o vacío para buscar en <code>/me/messages</code> sin restringir carpeta
<code>filter</code>	string	Expresión OData <code>\$filter</code> (ej: <code>sentDateTime ge ... and sentDateTime le ...</code>)
<code>search</code>	string	Expresión <code>\$search</code> (ej: <code>"subject:factura"</code>)
<code>top</code>	number	Máximo de mensajes a traer (<code>\$top</code>)
<code>run_id</code>	number	FK hacia <code>mailing.fetch_runs</code> , para trazabilidad
<code>is_sent</code>	boolean	Si <code>true</code> , los mensajes se guardan marcados como enviados (<code>mailing.messages.is_sent</code>)

Nodos, en orden:

1. **When Executed by Another Workflow** — `n8n-nodes-base.executeWorkflowTrigger` Trigger del subworkflow. Declara los 6 parámetros de entrada de la tabla de arriba (`workflowInputs`). No hace nada más; solo expone `$json.folder` , `$json.filter` , etc. al resto del workflow.
2. **Graph: List Messages** — `n8n-nodes-base.httpRequest` GET contra Microsoft Graph, usando la credencial `MailingAI Graph OAuth2` (`microsoftOAuth2Api`).
 - URL: si `folder` viene con valor, pega contra `https://graph.microsoft.com/v1.0/me/mailFolders/{folder}/messages` ; si viene vacío, pega contra `https://graph.microsoft.com/v1.0/me/messages` (todas las carpetas).
 - Query params: `$filter` , `$search` , `$top` (default 50 si no viene), `$orderby=sentDateTime desc` .
 - Devuelve el objeto crudo de Graph, con el array de mensajes en `value` .
3. **Split Out Messages** — `n8n-nodes-base.splitOut` Separa el array `value` de la respuesta de Graph en un item de n8n por mensaje (de "1 item con un array" pasa a "N items", uno por correo).
4. **Map To mailing.messages** — `n8n-nodes-base.set` Traduce cada mensaje de Graph al esquema de la tabla `mailing.messages` : `message_id` (`id` de Graph), `conversation_id` , `internet_message_id` , `subject` , `from_address` / `from_name` (desde `from.emailAddress`), `to_addresses` / `cc_addresses` (arrays de direcciones, serializados a JSON string), `sent_datetime` , `received_datetime` , `has_attachments` , `importance` , `categories` , `body_preview` , `web_link` , y `raw_record` (el mensaje completo de Graph, como JSON string, para no perder nada). También toma `run_id` e `is_sent` del trigger original (nodo 1), no del mensaje de Graph.
5. **Upsert mailing.messages** — `n8n-nodes-base.postgres` `INSERT ... ON CONFLICT (message_id) DO UPDATE` contra `mailing.messages` , usando la credencial `MailingAI Postgres` . Si el mismo correo ya existía (mismo `message_id`), lo actualiza en vez de duplicarlo — por eso volver a traer el mismo rango de fechas no genera filas repetidas.
6. **Aggregate Results** — `n8n-nodes-base.aggregate` Junta todos los items (uno por mensaje procesado) de vuelta en un solo item, metiendo el array completo en el campo `messages` . Es el paso previo para poder contar cuántos mensajes se procesaron.
7. **Return Summary** — `n8n-nodes-base.set` Calcula `fetches_count = messages.length` y lo deja como única salida del subworkflow. Es lo que reciben los workflows 01/02/03 en `$json.fetches_count` cuando llaman a este subworkflow con `Execute Workflow` .

01 — Fetch Sent Items

Archivo: `01-mailingai-fetch-sent-items.json`

Caso de uso más simple: traer los correos de la carpeta **Enviados** de los últimos 30 días (parametrizable). Es el workflow recomendado para probar la conexión con Graph por primera vez.

Nodos, en orden:

1. **Manual Trigger** — `n8n-nodes-base.manualTrigger` Arranque manual (botón "Execute Workflow" en el editor).
 2. **Set: Parametros** — `n8n-nodes-base.set` Define los 3 parámetros editables del workflow: `date_from` (default: hoy - 30 días, ISO), `date_to` (default: ahora, ISO), `top` (default: 50). Para cambiar el rango o el límite de mensajes, se edita este nodo.
 3. **Postgres: Insert fetch_runs** — `n8n-nodes-base.postgres` Inserta una fila en `mailing.fetch_runs` con `folder='sentitems'`, las fechas y el `top` elegidos, y `status='started'`. Devuelve el `run_id` generado (columna de salida configurada en `options.outputColumns`), que se usa para todo el resto del workflow.
 4. **Execute: Graph Fetch** — `n8n-nodes-base.executeWorkflow` Llama al subworkflow 00 con `folder="SentItems"`, `filter` construido como `sentDateTime ge {date_from} and sentDateTime le {date_to}`, `search=""`, `top` y `run_id` tomados de los nodos anteriores, `is_sent=true`. Esto ejecuta todo el flujo de Graph → normalización → upsert descrito arriba.
 5. **Postgres: Update fetch_runs** — `n8n-nodes-base.postgres` Actualiza la fila de `fetch_runs` creada en el paso 3: `status='success'`, `finished_at=ahora`, `total_messages` = el `fetches_count` que devolvió el subworkflow.
-

02 — Fetch Message Series (parametrizado)

Archivo: `02-mailingai-fetch-message-series.json`

Igual que 01, pero con filtros libres: remitente, texto en el asunto, carpeta y rango de fechas, en vez de un flujo fijo para Enviados.

Nodos, en orden:

1. **Manual Trigger** — `n8n-nodes-base.manualTrigger`
 2. **Set: Parametros** — `n8n-nodes-base.set` Define 6 parámetros editables: `folder` (vacío = todas las carpetas), `from_address` (vacío = cualquier remitente), `subject_contains` (vacío = sin filtro de texto), `date_from`, `date_to` (defaults: últimos 30 días), `top` (default 50).
 3. **Set: Build Graph Query** — `n8n-nodes-base.set` Arma la query de Graph a partir de los parámetros:
 - `graph_filter`: junta con `and` las condiciones que tengan valor (`from/emailAddress/address eq '...'`, `sentDateTime ge ...`, `sentDateTime le ...`), descartando las vacías.
 - `graph_search`: si `subject_contains` tiene valor, arma `"subject:{texto}"`; si no, queda vacío.
 - Usa `options.includeOtherFields: true` para no perder los parámetros del nodo anterior.
 4. **Postgres: Insert fetch_runs** — `n8n-nodes-base.postgres` Inserta en `mailing.fetch_runs` con `folder` (o `'messages'` si vino vacío), fechas, `search_query`, `filter_description` y `top_requested`, `status='started'`. Devuelve `run_id`.
 5. **Execute: Graph Fetch** — `n8n-nodes-base.executeWorkflow` Llama al subworkflow **00** con `folder`, `filter` y `search` calculados en el paso 3, más `top` y `run_id`. `is_sent` se calcula solo: `true` únicamente si `folder` (en minúsculas) es `sentitems`.
 6. **Postgres: Update fetch_runs** — `n8n-nodes-base.postgres` Igual que en el workflow 01: marca `status='success'`, `finished_at`, `total_messages`.
-

03 — Fetch Related Thread

Archivo: `03-mailingai-fetch-related-thread.json`

Dado el `conversation_id` de un correo ya guardado en `mailing.messages` (columna que identifica el hilo/thread en Graph), trae **todos** los mensajes de esa conversación, estén en la carpeta que estén.

Nodos, en orden:

1. **Manual Trigger** — `n8n-nodes-base.manualTrigger`
 2. **Set: Parametros** — `n8n-nodes-base.set` Define `conversation_id` (sin default real — trae el placeholder `REEMPLAZA_CON_UN_CONVERSATION_ID_DE_mailing.messages`, hay que pegar un valor real antes de ejecutar, por ejemplo con `SELECT DISTINCT conversation_id FROM mailing.messages; )` y `top` (default 100).
 3. **Postgres: Insert fetch_runs** — `n8n-nodes-base.postgres` Inserta en `mailing.fetch_runs` con `folder='related'`, `filter_description = conversationId eq '{conversation_id}'`, `top_requested`, `status='started'`. Devuelve `run_id`.
 4. **Execute: Graph Fetch** — `n8n-nodes-base.executeWorkflow` Llama al subworkflow **00** con `folder=""` (busca en `/me/messages`, sin restringir carpeta), `filter=conversationId eq '{conversation_id}'`, `search=""`, `top`, `run_id`, `is_sent=false` (los mensajes de un hilo pueden venir de cualquier carpeta, no se asume que son enviados).
 5. **Postgres: Update fetch_runs** — `n8n-nodes-base.postgres` Igual que en 01/02: marca `status='success'`, `finished_at`, `total_messages`.
-

04 — Generate Activity Charts

Archivo: `04-mailingai-generate-activity-charts.json`

Genera un PNG (línea de tiempo o histograma) a partir de lo que ya haya en `mailing.messages`, llamando al backend FastAPI (`/charts/timeline` o `/charts/histogram`). Tiene dos ramas paralelas — una por tipo de gráfico — que un nodo `IF` decide cuál ejecutar.

Nodos, en orden:

1. **Manual Trigger** — `n8n-nodes-base.manualTrigger`
2. **Set: Parametros** — `n8n-nodes-base.set` Define `chart_type` (`"timeline"` o `"histogram"`, default `"timeline"`), `date_from` y `date_to` (default: últimos 30 días, formato `yyyy-LL-dd`, usados solo por la rama de línea de tiempo).
3. **IF: Chart Type** — `n8n-nodes-base.if` Evalúa `chart_type === "timeline"`. Salida `true` (índice 0) → rama de línea de tiempo. Salida `false` (índice 1) → rama de histograma.

Rama línea de tiempo (`chart_type = "timeline"`)

4. **Postgres: Query by_day** — `n8n-nodes-base.postgres` `SELECT day, message_count FROM mailing.v_messages_by_day WHERE day BETWEEN {date_from} AND {date_to} ORDER BY day;` — una fila por día con actividad.
5. **Aggregate: Timeline Rows** — `n8n-nodes-base.aggregate` Junta todas las filas en un solo item, en el campo `rows`.
6. **Set: Build Timeline Payload** — `n8n-nodes-base.set` Arma el body para el backend: `title` fijo ("Actividad de correo por dia") y `points` = `rows` mapeado a `{date, count}` (serializado a JSON string).
7. **Backend: POST /charts/timeline** — `n8n-nodes-base.httpRequest` `POST` `http://backend:8000/charts/timeline` con `{title, points}` como body JSON. La respuesta se pide como archivo binario (`responseFormat: "file"`, queda en la propiedad `data` del item) — es el PNG generado por matplotlib.
8. **Write: Timeline PNG** — `n8n-nodes-base.writeBinaryFile` Escribe el binario recibido en `/files/maillingai/out/timeline-{fecha-hora}.png` (visible en el host en `share/maillingai/out/`).
9. **Postgres: Insert chart_runs (timeline)** — `n8n-nodes-base.postgres` Inserta en `mailing.chart_runs`: `chart_type='timeline'`, `params` (rango de fechas usado, como JSON), `output_file` (la ruta del PNG escrito en el paso anterior).

Rama histograma (`chart_type = "histogram"`)

- 4'. **Postgres: Query by_sender** — `n8n-nodes-base.postgres` `SELECT from_address AS label, message_count FROM mailing.v_messages_by_sender ORDER BY message_count DESC LIMIT 20;` — top 20 remitentes por cantidad de correos.
 - 5'. **Aggregate: Histogram Rows** — `n8n-nodes-base.aggregate` Igual que en la otra rama: junta las filas en un item, campo `rows`.
 - 6'. **Set: Build Histogram Payload** — `n8n-nodes-base.set` Arma el body: `title` fijo ("Correos enviados por remitente") y `buckets` = `rows` mapeado a `{label, count}`.
 - 7'. **Backend: POST /charts/histogram** — `n8n-nodes-base.httpRequest` `POST` `http://backend:8000/charts/histogram` con `{title, buckets}`. Misma configuración de respuesta binaria que la rama de línea de tiempo.
 - 8'. **Write: Histogram PNG** — `n8n-nodes-base.writeBinaryFile` Escribe en `/files/maillingai/out/histogram-{fecha-hora}.png`.
 - 9'. **Postgres: Insert chart_runs (histogram)** — `n8n-nodes-base.postgres` Inserta en `mailing.chart_runs` con `chart_type='histogram'`, mismos `params`, `output_file` del histograma.
-

Credenciales que usan estos workflows

- **MailingAI Graph OAuth2** (`microsoftOAuth2Api`) — usada por el único nodo `Graph: List Messages` , dentro del subworkflow 00.
- **MailingAI Postgres** (`postgres`) — usada por todos los nodos `Postgres: *` en los 5 workflows.

Ambas quedan pre-enlazadas por `id` cuando se importan con `scripts/import-n8n.ps1` / `n8n/import.sh` (ver README de la raíz del proyecto).